本章路線

這章要回答的核心問題是：作業系統如何把連續的 logical address 安全映射到有限且分散的 physical memory？

前置知識。 二進位位址、暫存器與 memory bus、process 隔離。

- 推導 base/limit、segment、page 的轉換
- 比較 internal/external fragmentation
- 分析 TLB 命中率與 page table 層級

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

位址保護：先檢查，再重定位

在單一連續區段模型中，base 是該行程實體區段的起點，limit 是區段長度。CPU 產生 offset d 後，硬體先檢查 `0 ≤ d < limit`；通過才計算 physical address `base + d`，否則觸發保護例外。base 與 limit 的設定由特權程式控制。

例如 base=`0x8000`、limit=`0x2000`，d=`0x1FFF` 的實體位址是 `0x9FFF`；d=`0x2000` 正好等於長度，必須拒絕。若一次存取跨越多個位元組，整個範圍都要在界內，不能只檢查起始位址。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Swapping

- swapping 把整個 process 移到 backing store，以釋放主記憶體；需要可容納 process 的連續或可映射空間與恢復狀態。
- 若每個 page 都可選擇性載入，demand paging 比 whole-process swapping 細緻；成本與政策不同。

例如 80 KiB 的行程換出至可用空間 100 KiB 的 backing store，須保存必要狀態；若 DMA 仍使用某個 buffer，須等待完成或採 pin 等方式防止其 frame 被回收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

連續配置：總空間足夠也可能放不下

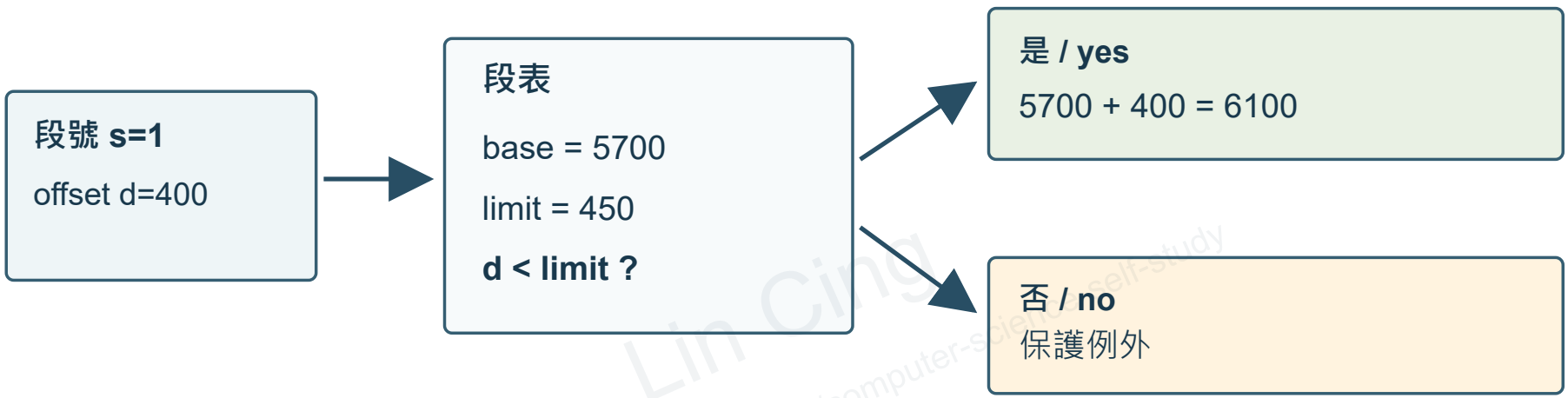
空洞依位址順序為 100、500、200 KiB；新的連續需求為 180 KiB。first-fit 選第一個足夠的 500，留下 320；best-fit 選 200，留下 20；worst-fit 選最大的 500，留下 320。三種策略都是針對同一初始狀態，各自獨立試配。

另一狀態有四個分離的 5 KiB 空洞。總共雖有 20 KiB，仍沒有單一區段容納 10 KiB：這是 **external fragmentation**。若使用 4 KiB 配置單位滿足 6 KiB 需求，配置 8 KiB、其中 2 KiB 未使用，則屬 **internal fragmentation**。壓縮需要能重新定位，且必須協調正在進行的 I/O 和位址引用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Segmentation：每個段都有自己的界線

段位址的保護檢查



offset=449 合法；offset=450 超界

指定 segment 0 的 base=1000、limit=339，segment 1 的 base=5700、limit=450，segment 2 的 base=6300、limit=1280；數值均為 byte。`<0,330>` 因 `330<339` 合法，轉為 1330；`<1,400>` 轉為 6100；`<2,1380>` 因 `1380≥1280` 觸發例外。

Segmentation：每個段都有自己的界線 （續）

limit 是長度而非最後合法 offset：segment 1 的 offset 449 合法，450 不合法。不同 segment 可以有不同長度、權限和實體起點；同一段內則依此模型使用連續實體區間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

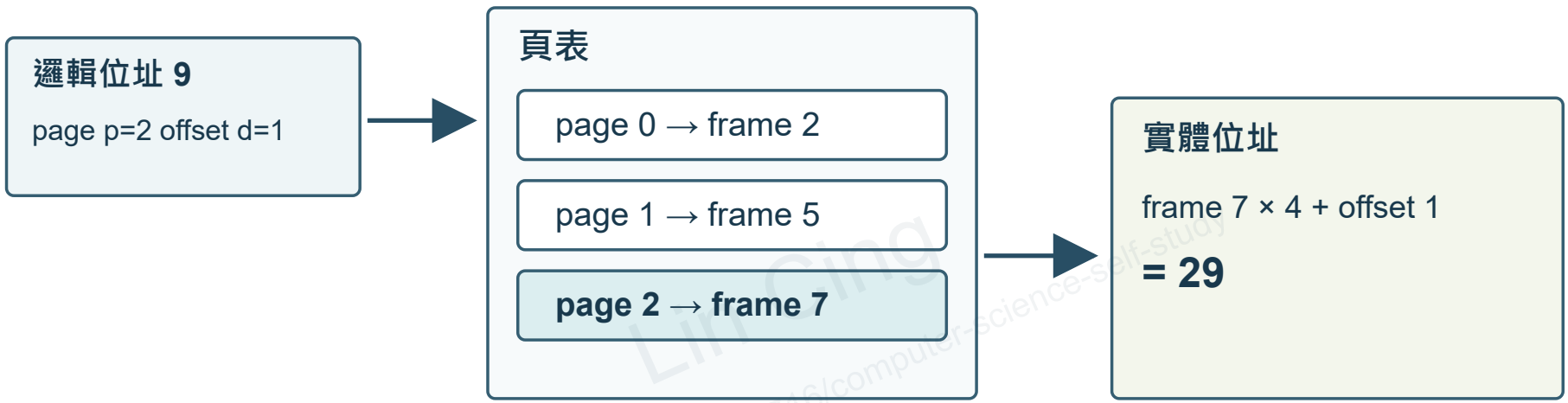
Paging

- logical address 分成 page number p 與 offset d。page table[p] 給 frame number f，實體位址為 f×page_size+d。
- page 大小是 trade-off。較小頁減少 internal fragmentation，但增大 table 與轉換 overhead；較大頁降低項目數，卻可能浪費每個物件的尾端空間。



一次頁面轉換：從位址 9 到 29

4-byte 頁面：先分解，再查表



offset 不變；頁號經頁表轉為 frame 號

page size = 4 bytes。對 logical address 9，先算 `p = floor(9/4) = 2`、`d = 9 mod 4 = 1`；查表得到 frame `f = 7`，因此 physical address 為 `7×4+1 = 29`。offset 原樣保留；page 與 frame 的編號不必相等。

一次頁面轉換：從位址 9 到 29 （續）

本例只列有效的 page 0、1、2；若查不到有效的 page-table entry，不能依頁號猜 frame，而應依系統規則產生 fault 或處理缺頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

位址位元與記憶體碎片

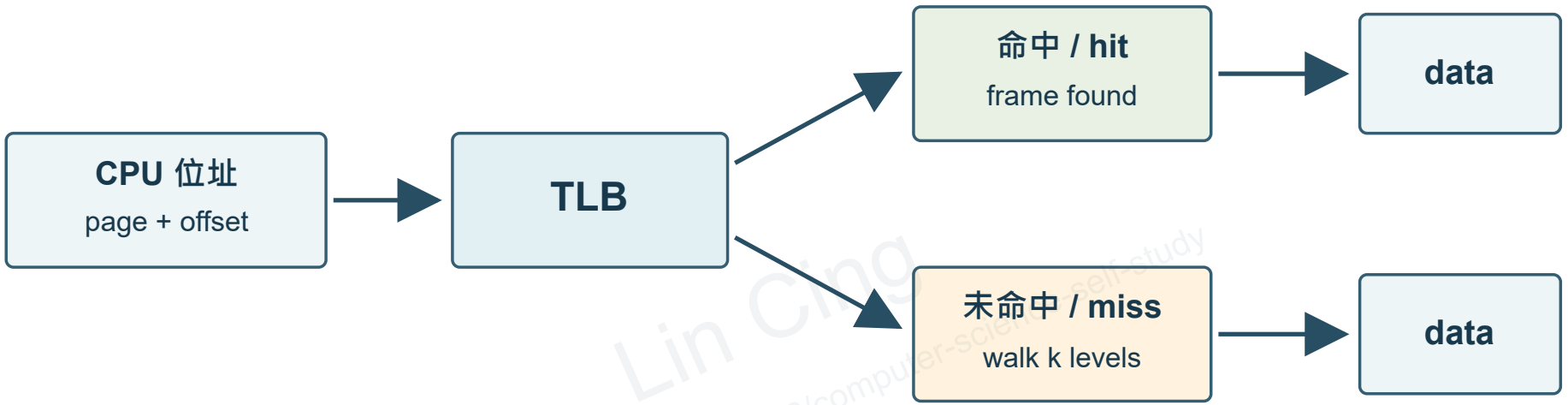
每頁 1 KiB = 2^{10} bytes、每個 address space 有 8 = 2^3 pages，則 logical address 需要 3 個 page bits 加 10 個 offset bits，共 **13 bits**。若實體記憶體有 32 = 2^5 frames，physical address 需要 5 個 frame bits 加同樣的 10 個 offset bits，共 **15 bits**。假設位址以 byte 為單位，且兩種位址空間均使用完整編碼範圍。

用 10 KB 的頁框配置一段 32 KB 的連續邏輯區域，需要 4 頁、分配 40 KB；若前三頁填滿，最後一頁用 2 KB，內部碎片為 8 KB。這裡 KB 代表 1000 bytes，僅用於容量算例；二進位位址拆分的 1 KiB 則是 1024 bytes。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

TLB 命中與未命中如何計時？

TLB：命中與未命中的記憶體存取路徑



串列模型：hit = TLB + data；miss = TLB + k 次表項讀取 + data

TLB hit 時取得 frame 後訪問資料；miss 時需要查 page table，再訪問資料。以下採串列查找、各次記憶體訪問成本相同、固定頁表層數，且不計 cache、page-walk cache、替換與 page fault。

TLB 命中與未命中如何計時？ (續)

一層頁表時，TLB lookup = 10 ns，RAM access = 200 ns，hit rate = 0.8。hit 成本 `10+200=210 ns`；miss 成本 `10+200+200=410 ns`。因此 EAT = `0.8×210 + 0.2×410 = 250 ns`。這是此模型下的平均值，不是所有 CPU 的實測延遲。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

頁表多一層，miss 成本怎麼變？

改用兩層頁表、TLB lookup = 20 ns、RAM access = 200 ns、hit rate = 0.8；其他假設不變。hit 是 `20+200=220 ns`；miss 先後讀兩層 entry 和資料，為 `20+2×200+200=620 ns`。

所以 EAT = `0.8×220 + 0.2×620 = 300 ns`。比較兩個數例時，同時改了 TLB lookup 時間和頁表層數；不能把 50 ns 的差距全部歸因於多一層頁表。實機也可能平行查找或快取中間表項。

一般而言，hit rate 為 h、TLB 時間 t、每次 memory 時間 m、miss 查 k 層時，EAT = h(t+m)+(1−h)(t+km+m)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

保護與共享

- 每個 page table entry 可含 valid、read/write、execute、user/kernel 等位元；另一 process 只有在其 page table 映射同一 frame 時才可共享。
- 共享唯讀 code 可節省記憶體，但寫入或可執行權限若設錯，就會破壞隔離。

例如 P1 與 P2 的頁表各映射同一實體 frame，且兩邊均為 user-readable、read-only；兩者可讀取同一內容，user write 則被拒。移除其中一個映射後，另一個仍可存在；回收前須確認所有引用及 TLB 失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

兩層頁表：空間取決於實際映射

32-bit byte address、4 KiB pages、每筆 entry 4 bytes。一張 4 KiB 表可放 $1024 = 2^{10}$ 筆 entry；位址分成 `10-bit root index | 10-bit leaf index | 12-bit offset`。每張 leaf table 覆蓋 1024 頁，即 4 MiB 虛擬位址。

映射一頁時，若根表必須完整配置且恰好只需一張 leaf，表格本身為 `4 KiB + 4 KiB = 8 KiB`；資料頁不含在內。全部 2^{20} 個虛擬頁都映射時，需要 1024 張 leaf，合計 `4 KiB + 1024×4 KiB = 4 MiB + 4 KiB`。稀疏配置節省未使用的 leaf，不能把 8 KiB 當成所有兩層頁表的最大值。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倒排頁表：按 frame 而非虛擬頁計數

若實體記憶體為 1 GiB、page/frame 為 8 KiB，則 frame 數為 $2^{30} / 2^{13} = 2^{17}$。假設每個 frame 的記錄恰好 4 bytes，這些記錄共 $2^{17} \times 4 = 2^{19}$ bytes = 512 KiB。

這只是依 frame 編號排列的記錄空間；用來加速以 `(address-space ID, virtual page)` 查詢的 hash/index、碰撞鏈結及共享 frame 的多重映射紀錄都可能額外占空間。記錄格式亦須足以保存所需識別與權限，4 bytes 是本例假設，不是通用實作大小。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IA-32 protected mode：從 linear address 到 page entry

本模型固定 IA-32 protected mode、`CR0.PG=1`、`CR4.PAE=0` 與 4 KiB page；輸入是已通過 segment descriptor 檢查的 32-bit linear address。segment selector 先用 LDT/GDT descriptor 的 base 與 limit 形成 linear address，paging unit 再把 linear address 轉成 physical address。因此 `0x12345678` 是 linear address，不把它誤稱為尚未經過 segmentation 的 logical address。

每張 IA-32 page directory 與 page table 有 $4096/4 = 1024$ entries，索引各 10 bits。令 linear address 為 L：

$$\mathrm{PDI} = (L \gg 22)\&0x3FF, \quad \mathrm{PTI} = (L \gg 12)\&0x3FF, \quad \mathrm{offset} = L\&0xFFF.$$

固定 CR3 page-directory base=`0x5000`、下一層 page-table base=`0x6000`。entry address 由 base+index×4 得到：

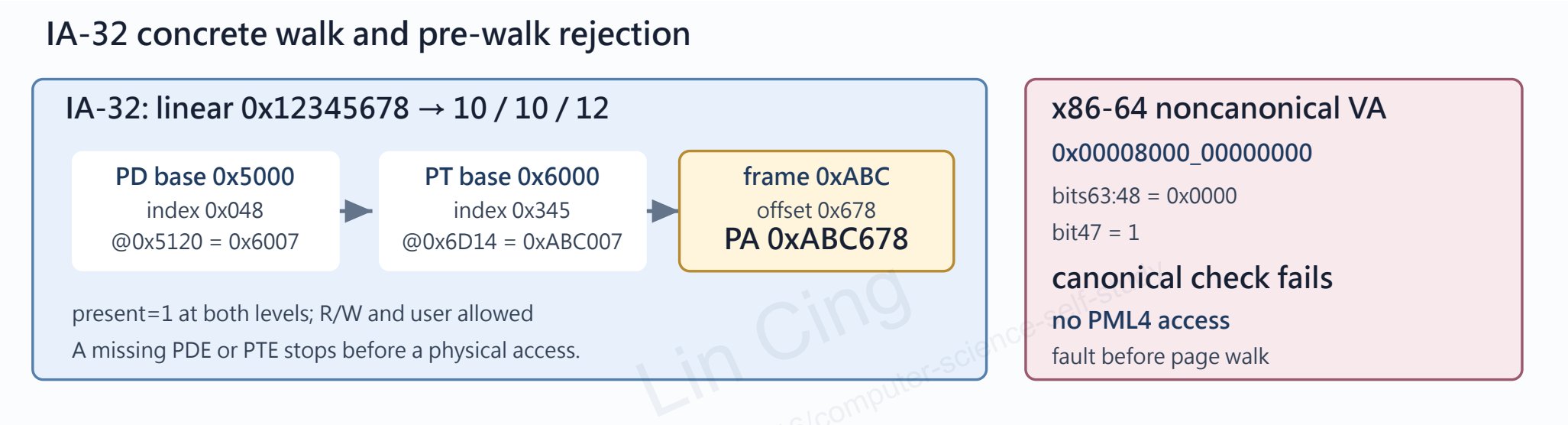
層級	index	entry 位址	表項內容	解讀
page directory	`0x048`	`0x5000+4×0x048=0x5120`	`0x00006007`	present；next PT base=`0x6000`；R/W、user 允許
page table	`0x345`	`0x6000+4×0x345=0x6D14`	`0x00ABC007`	present；4 KiB frame=`0xABC`；R/W、user 允許

IA-32 protected mode：從 linear address 到 page entry （續）

因此 offset=`0x678`，physical address=`0xABC000+0x678=0xABC678`。這一路徑使用 page directory、page table 與目標 page 三個 4 KiB 記憶體頁；table 本身的 page-table 配置成本另計，但由上表的 entries 可完全重建。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IA-32 protected mode：從 linear address 到 page entry （續）



若 PDE 的 present=0，paging unit 在讀下層 PT 前就 fault；若 PTE 的 present=0，不能把 frame 欄位解成有效記憶體。若 CPU 支援 4 MiB page 且 `CR4.PSE=1`，PDE 的 `PS=1` 會讓 PDI 之後的 22 bits 成為 offset，直接跳過 page table；這是不同的 encoding。若 `CR4.PSE=0`，32-bit paging 忽略 PDE 的 PS 位，改把該 PDE 當作指向 4 KiB page table，不能當成 4 MiB 映射。

x86-64 canonical address 與四層頁表

OSC 9e 的 x86-64 案例使用 48-bit virtual address 與四層 4 KiB paging。有效位元為 `[47:39]`、`[38:30]`、`[29:21]`、`[20:12]`，每層 9-bit index；offset 為 `[11:0]`。每張 table 有 $4096/8 = 512$ entries。

本頁固定48-bit canonical模式，輸入先限定0≤v<2^64。Canonical condition要求bit47複製到所有`[63:48]`，不能只比較bit63與bit47：

$$\text{canonical}(v) = \left((v \gg 48) = \begin{cases} 65535, & ((v \gg 47)\ \&\ 1) = 1, \\ 0, & ((v \gg 47)\ \&\ 1) = 0. \end{cases} \right)$$

反例`v=0x0001_0000_0000_0000=2^48`的bit63=bit47=0，但bits63:48=1而非0，故noncanonical，不能walk。合法lower-half位址若單獨把任一bit48…63翻成1，都破壞sign extension；合法upper-half若把任一對應bit翻成0亦然。四個邊界為0與0x00007FFFFFFFFFFF合法、0x0008000000000000至0xFFF7FFFFFFFFFFFF非法、0xFFF8000000000000至0xFFFFFFFFFFFFFFFF合法。

固定 CR3=`0x1000`，沿路 table bases 為 PDPT=`0x2000`、PD=`0x3000`、PT=`0x4000`。目標 VA=`0x00007FFF_FFFFF123` 的 bit47=0、bits63:48=0，所以 canonical。各層都提 9 bits：

x86-64 canonical address 與四層頁表 （續）

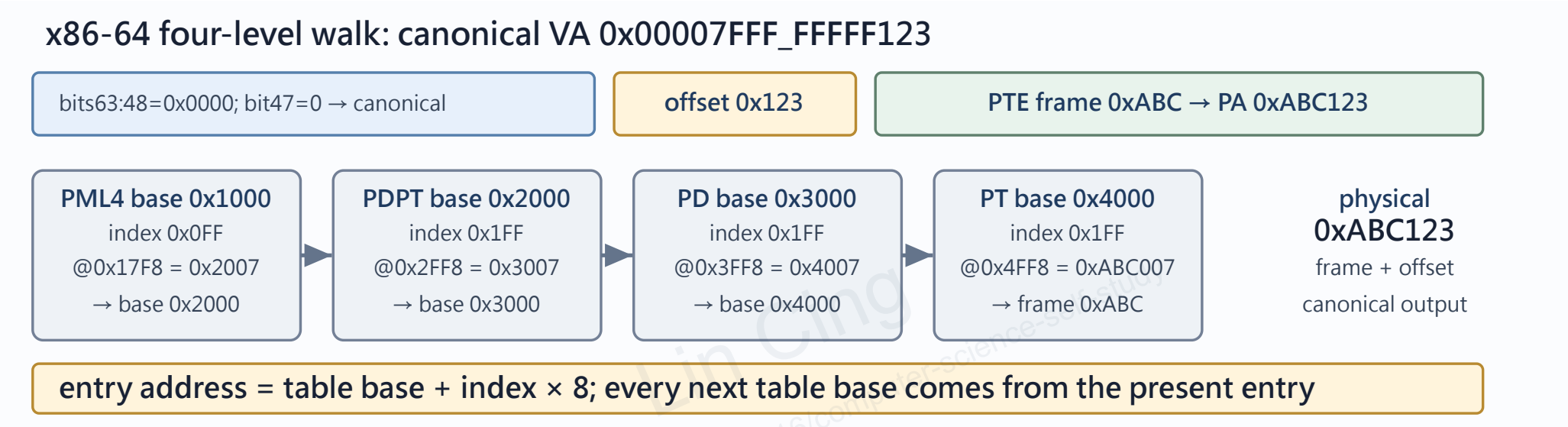
層級	index	entry offset	entry 位址	entry 值	下一個 frame/table
PML4	`0x0FF`	`0x7F8`	`0x17F8`	`0x2007`	PDPT base `0x2000`
PDPT	`0x1FF`	`0xFF8`	`0x2FF8`	`0x3007`	PD base `0x3000`
PD	`0x1FF`	`0xFF8`	`0x3FF8`	`0x4007`	PT base `0x4000`
PT	`0x1FF`	`0xFF8`	`0x4FF8`	`0xABC007`	data frame `0xABC`

entry$_\ell$ 位址的一般式是

$$address_\ell = base_\ell + index_\ell \times 8.$$

最後 offset=`0x123`，故 physical address=`0xABC000+0x123=0xABC123`。若只算 indices，仍不足以重建表格；本例另給每層 base、entry 位址與表項值，任何一步都可從上表重新展開。

x86-64 canonical address 與四層頁表 （續）



2 MiB 與 1 GiB large-page encodings 會在較上層終止 walk，並把較多低位址 bits 當 offset。它們需要不同 entry format 與 alignment；本頁的 9/9/9/9/12 只描述指定的 4 KiB 路徑。

合法、非法與權限 fault 的完整分流

本頁固定四層 48-bit x86-64 paging、user-mode `CPL=3`，CPU 支援 NX 且 `IA32_EFER.NXE=1`；SMEP、SMAP、PKE 等額外保護均不介入。canonical、present、U/S、R/W 與 NX 回答不同問題。canonical 失敗時不產生 page-table memory access；canonical 通過後，page walk 先找 mapping，再依操作合併權限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

合法、非法與權限 fault 的完整分流 （續）

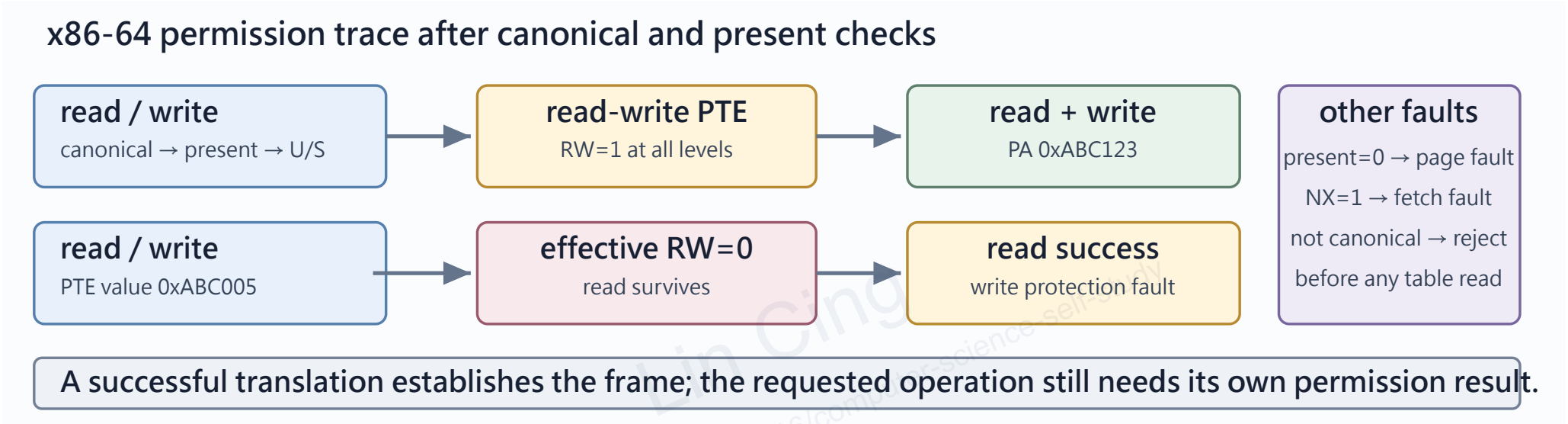
案例	canonical	mapping	effective permission	read	write	execute
本頁正常路徑	true	四層 present	全層 U/S=1、R/W=1、NX=0	`0xABC123`	`0xABC123`	允許
PTE 改為 `0xABC005`	true	四層 present	全層 U/S=1；R/W=0；NX=0	`0xABC123`	protection fault	允許
本次PML4[0x0FF] absent	true	present=0	不成為有效映射	page fault	page fault	instruction page fault
PTE 加上 NX	true	四層 present	R/W 允許；NX=1	資料讀取成功	資料寫入成功	instruction fetch fault
`0x00008000_00000000`	false	不應 walk	無法定義	reject	reject	reject

合法、非法與權限 fault 的完整分流 （續）

canonical check 不可取代 U/S；user 存取要求沿途 entry 的 user 權限允許。本例 user write 需要 effective R/W=1；supervisor write 另受 `CR0.WP` 等條件影響，不可直接套用。NX 區分資料讀取與 instruction fetch；若 `EFER.NXE=0`，表項的 XD/NX bit 為保留位，將其設為 1 會造成 reserved-bit page fault，不能只視為禁止執行。root 高位、table entry 與 page frame 都存在，也不等於任意操作有權限。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

合法、非法與權限 fault 的完整分流 （續）



這個模型固定 48-bit 4-level 設定。PAE、large pages、5-level paging 或不同 physical-width 會改變欄位與 entry 數，必須換成對應模式；不能把本頁的 masks 當成所有 Intel CPU 的通則。TLB 有效期則由 entry 更新與必要的 invalidation 決定。

ARMv7-A AArch32：short-descriptor 的兩層 walk

本頁固定 ARMv7-A、AArch32、short-descriptor translation format、`TTBCR.N=0`、`TTBCR.PD0=0`、`SCTLR.AFE=0`、PL1 建表及 4 KiB small pages。P1、P2 使用不同且尚未重用的 ASID，例如 1、2。這段不是 AArch64 long descriptor。

short-descriptor 格式的 L1 table 有 4096 個 4-byte entries，總大小為 $4096 \times 4 = 16,384$ bytes，即 16 KiB。`TTBCR.N=0` 時所有輸入 VA 都由 TTBR0 選 root，並以 VA `[31:20]` 索引完整 L1 table；TTBR0 的 table base 因此必須 16 KiB 對齊。

$$L1_address = TTBR0 + ((VA \gg 20) \& 0xFFF) \times 4.$$

coarse L1 page-table descriptor 指向 L2 table。short-descriptor 的 L2 table 有 256 個 4-byte entries，總大小 1 KiB，base 只需 1 KiB 對齊；硬體使用 descriptor 的 `[31:10]` 作為 table base，因此不能任意插入非對齊位址。

$$L2_address = L2_base + ((VA \gg 12) \& 0xFF) \times 4, \qquad PA = frame + (VA \& 0xFFF).$$

ARMv7-A AArch32：short-descriptor 的兩層 walk （續）

本例的四個 table 區間完整且互不重疊：

storage	base	range	size	alignment check
P1 root	`0x8000`	`0x8000–0xBFFF`	16 KiB	`0x8000 % 0x4000=0`
P2 root	`0xC000`	`0xC000–0xFFFF`	16 KiB	`0xC000 % 0x4000=0`
P1 L2	`0x10000`	`0x10000–0x103FF`	1 KiB	`0x10000 % 0x400=0`
P2 L2	`0x10400`	`0x10400–0x107FF`	1 KiB	`0x10400 % 0x400=0`

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ARMv7-A AArch32：short-descriptor 的兩層 walk （續）

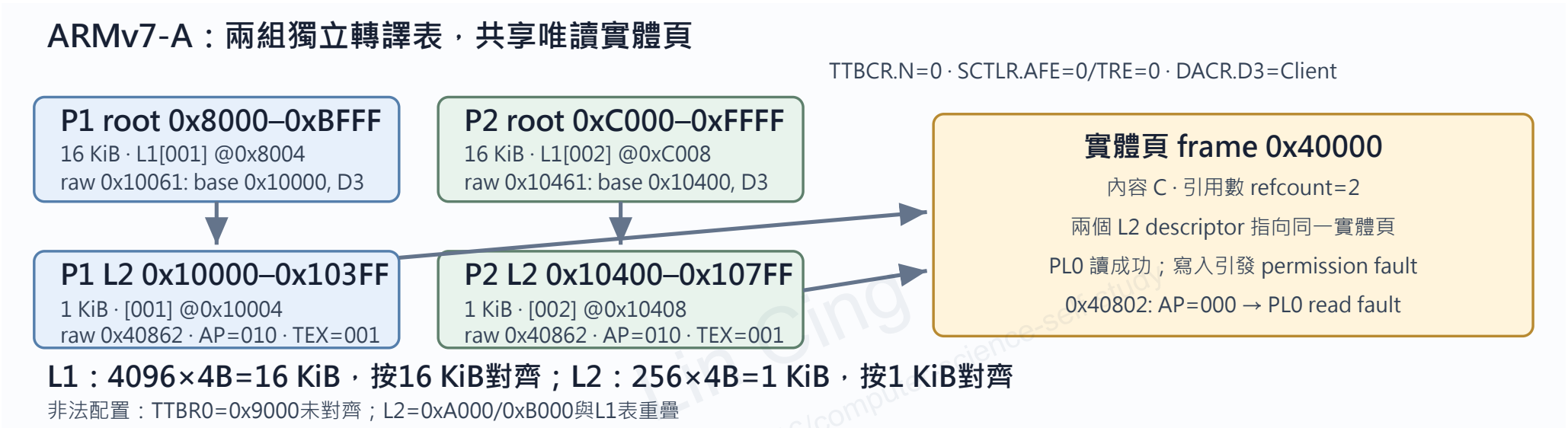
P1 VA `0x00101000` 與 P2 VA `0x00202000` 的實際位址為：

process	VA	L1 index → entry address	L2 base	L2 index → entry address	frame	PA
P1	`0x00101000`	`0x001→0x8004`	`0x10000`	`0x001→0x10004`	`0x40000`	`0x40000`
P2	`0x00202000`	`0x002→0xC008`	`0x10400`	`0x002→0x10408`	`0x40000`	`0x40000`
P1，非零 offset	`0x00101A5C`	`0x001→0x8004`	`0x10000`	`0x001→0x10004`	`0x40000`	`0x40A5C`

第三列的 page offset 是 VA `[11:0]=0xA5C`；它不參與任一層索引，會完整保留至輸出位址。只保留最低 8 位會錯把 PA 算成 `0x4005C`。

每個 address space 都保留完整 root 儲存。即使 VA 只使用少數 entries，也不能只保存那幾筆 descriptor；其餘 entries 必須有架構允許的 invalid 或其他有效內容。兩張 16 KiB roots 與兩張 1 KiB L2 tables 佔用不同的 physical ranges。

ARMv7-A AArch32：short-descriptor 的兩層 walk （續）



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

raw descriptors、domain、AFE 與 AP

本例在每個 L1 page-table descriptor 使用 domain 3，並設定 `DACR.D3=0b01`，即 Client。domain 只存在 L1 page-table descriptor；L2 entry 繼承該 domain。

當 `SCTLR.AFE=0` 時，使用 full AP[2:0] 模型。small page 的 AP[2] 在 bit9、AP[1:0] 在 bits[5:4]；不能套用 section descriptor 的 bits[11:10]。本例 L2 解碼後 `AP[2:0]=010`：PL1 read/write、PL0 read-only，所以 PL0 write 產生 Permission fault。

可從 raw word 直接解碼：

descriptor	raw word	解碼
P1 L1	`0x10061`	bits[1:0]=01 coarse；table base=`0x10000`；domain=3
P2 L1	`0x10461`	bits[1:0]=01 coarse；table base=`0x10400`；domain=3
P1/P2 L2	`0x40862`	frame=`0x40000`；type bits[1:0]=10；XN=0；nG=1；AP[2:0]=010；TEX=001、C=B=0

raw descriptors、domain、AFE 與 AP （續）

small-page descriptor 的 `field = (raw_word >> shift) & mask` 必須逐項套用：`AP2=(raw>>9)&1`、`AP1=(raw>>5)&1`、`AP0=(raw>>4)&1`、`TEX=(raw>>6)&7`、`C=(raw>>3)&1`、`B=(raw>>2)&1`、`XN=raw&1`。本例得到 AP010、TEX001、C=B=0、XN=0、nG=1。

`TEX=001,C=B=0` 在 `SCTLR.TRE=0` 下解為 Normal non-cacheable。instruction fetch 不能只由 XN=0 推出；本模型另固定 `SCTLR.WXN=UWXN=0`、mapping 可讀、domain Client，且沒有其他實作限制。若只改記憶體屬性為 strongly ordered，不能憑 XN=0 宣稱一定可直接執行。

同一個 frame 的 descriptor 可以給出不同權限。比較下列兩個設定：

raw word	正確解碼	結果
`0x40802`	AP[2:0]=000；TEX=0,C=B=0	AP 禁止 PL0 read/write；frame 位址有效也無法通過 read 權限檢查
`0x40022`	AP[2:0]=010；TEX=0,C=B=0	只示範 data AP；Strongly-ordered 不可藉 XN=0 推 instruction fetch

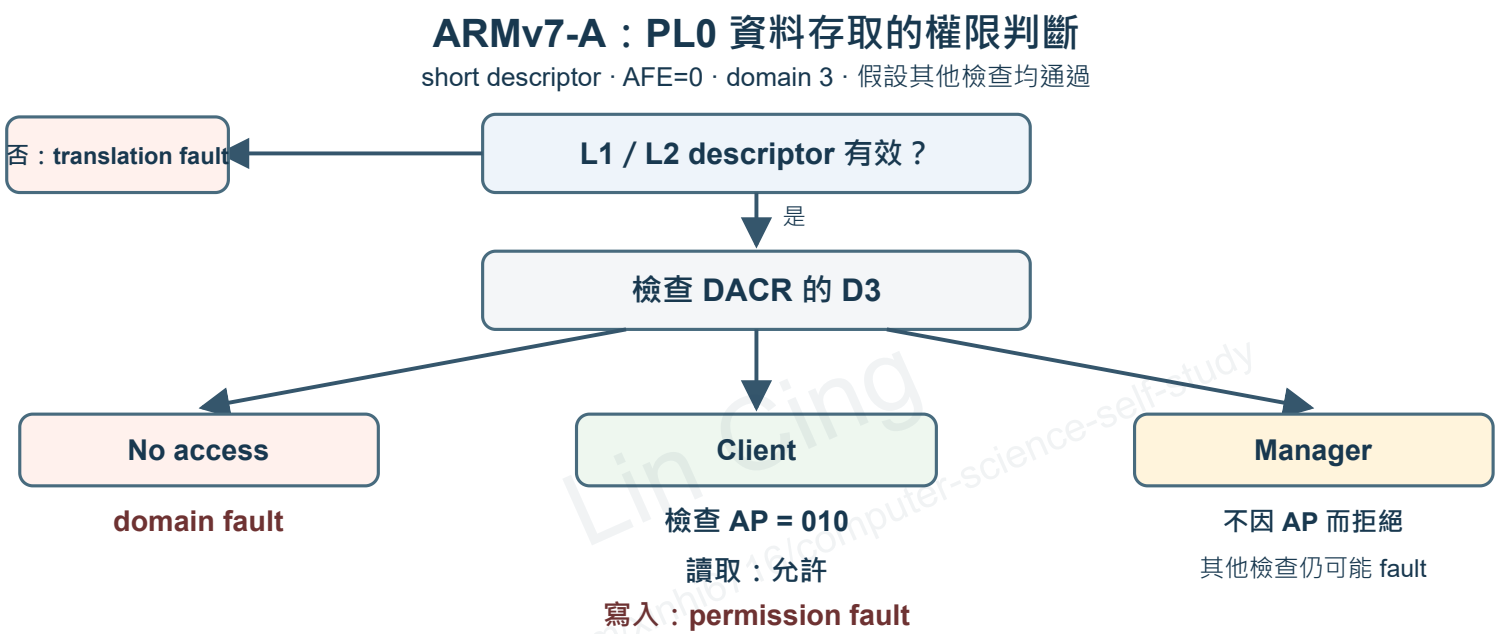
解碼時必須分別讀取 type、frame、AP 與記憶體屬性；只認出 frame 位址，尚不足以判定存取結果。

raw descriptors、domain、AFE 與 AP （續）

permission 不是只看 L2 word。DACR D3=Client 時才檢查 AP；若 D3=Manager（`0b11`），short-descriptor 的 AP 不被檢查，因此不能再用相同 L2 word 推出 write 必 fault。若 `SCTLR.AFE=1`，AP[0] 被重新定義成 Access flag，AP 模型改為 AP[2:1]；必須重新解碼 descriptor，不能沿用 AFE=0 的結論。

條件	PL0 read	PL0 write	理由
AFE=0、D3=Client、AP=010	成功	Permission fault	AP 對 PL0 授予 read、不授予 write
AFE=0、D3=No access	Domain fault	Domain fault	AP 檢查尚未成為成功證據
AFE=0、D3=Manager	不因 AP 被拒	不因 AP 被拒	Manager 不檢查 AP；其他架構 fault 另計
AFE=1、Client、同一字 `0x40862`	Access flag fault	Access flag fault	AP[0]=0；本例採軟體設定 Access flag
AFE=1、Client、改成 `0x40872`	成功	成功	AP[0]=1，AP[2:1]=01；此模型給 PL0 read/write，與 AFE=0 的 AP010 不同

raw descriptors、domain、AFE 與 AP （續）



表項與 domain 有效後才能判定權限；不要把位址轉換成功當作可寫。

權限 trace、mapping 移除與 TLB

假設 frame `0x40000` 僅有這兩筆映射，沒有其他引用或 pin；初始 mapping count=2。兩張 L2 entries 都是 `0x40862`，其 parent domains 都是 Client。P1、P2 分別使用尚未重用的不同 ASID 1、2，且 `TTBCR.PD0=0`。TLB 起初沒有這個 translation。

狀態與操作	translation	domain/AP	結果	frame 或 mapping count
S0：P1 read	`0x00101000→0x40000`	Client；AP=010 允許 PL0 read	success	C；mapping count 2
S0：P1 write	translation 有效	Client；AP=010 禁止 PL0 write	permission fault	C 不變
S0：P2 read	`0x00202000→0x40000`	Client；AP=010 允許 PL0 read	success	C；mapping count 2
S0：P2 write	translation 有效	Client；AP=010 禁止 PL0 write	permission fault	C 不變

權限 trace、mapping 移除與 TLB （續）

狀態與操作	translation	domain/AP	結果	frame 或 mapping count
S1：P1 descriptor absent	P1 L2 entry invalid，已完成對應 TLB maintenance	不再檢查資料 AP	translation fault	P2 仍映射；mapping count 1
S2：P2 read after P1 removal	獨立 P2 tables 仍有效	Client/read allowed	success	mapping count 1
S3：P2 descriptor absent	P2 L2 entry invalid，已完成對應 TLB maintenance	不再檢查資料 AP	translation fault	mapping count 0，所有舊 translation 已失效後 frame 才可回收

translation fault 與 permission fault 是不同證據。S1 的 P1 沒有效 mapping，因此不能說「頁存在只是不能寫」；S0 的 write 已有有效 mapping，且在 Client domain 下 AP 拒絕 PL0 write。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

權限 trace、mapping 移除與 TLB （續）

更新 descriptor 後，要依架構規則對所有可能持有舊 translation 的相關核心完成對應 VA／ASID 的 TLBI 與必要 barrier，確認失效完成後才能回收 frame。本例 nG=1，translation 屬於各自的 ASID；ASID 不決定 domain 或 AP。stale TLB entry 可能保留舊 translation 或權限，因此上表的 unmap 結果以失效操作完成為前提，不能在清除 PTE 後立即回收仍可能被舊 translation 存取的 frame。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

先驗 storage layout，再談共享 frame

下列配置即使能算出 entry 位址，仍不構成合法硬體配置：

錯誤配置	問題	判定
P2 TTBR0=`0x9000`，N=0	`0x9000%0x4000=0x1000`，未達 16 KiB 對齊	invalid root
P1 root=`0x8000`、P2 root=`0x9000`	ranges `0x8000–0xBFFF` 與 `0x9000–0xCFFF` 重疊	invalid ownership
P1 L2=`0xA000`	落在 P1 root `0x8000–0xBFFF` 內	table storage 重疊
P2 L2=`0xB000`	同樣落在 P1 root 內	table storage 重疊
只為 16 KiB L1 配置 4 KiB	4096 個 4-byte entries需要完整 16 KiB	storage 不足

修正後，P1/P2 各有獨立 roots 與 L2 tables，只有最後的 `frame 0x40000` 被共享。page identity、各層 storage、domain、AP、XN、frame reference count 與 TLB state 必須一起核對。

先驗 storage layout，再談共享 frame （續）

若 P1 descriptor 真的可寫，P1 的 write 會修改 P2 看見的共享 frame；這是 shared writable page 的語意，不是 read-only code。若系統要讓 P1 修改而 P2 保留舊內容，應使用 copy-on-write 或新 frame，不能只改其中一份 descriptor。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章收束

- page mapping 與權限是同一個 entry 契約的兩部分
- internal/external fragmentation 必須分類
- EMAT 是模型，需列出 cache 與 fault 假設

條件檢查。 使用本章任何模型前，先問它的假設是否仍成立。尤其是排程負載、同步保證、記憶體引用模式與硬體錯誤模型。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 8. Intel 64 and IA-32 SDM, Vol. 3A; Arm Architecture Reference Manual ARMv7-A/R.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study